Handling HTTP Requests in Node.js

Definition: HTTP requests are messages sent by a client, such as a web browser, to a server to request resources or perform operations. Node.js handles HTTP requests using the built-in http module, which allows developers to create servers and send responses to clients.

1. Creating an HTTP Server: The http.createServer() method creates an HTTP server and provides a callback function that receives the request and response objects.

2. Request Object (req): The request object contains information about the client's request.

- `req.url:` Returns the requested URL path.
- `req.method:` Returns the HTTP method, such as GET or POST.
- `req.headers:` Contains the request headers.

3. Response Object (res)

The response object is used to send data back to the client.

- res.writeHead(): Sets the HTTP status code and response headers.
- res.write(): Sends response data.
- res.end(): Ends the response.

4. Example Program

```
const http = require('http');
const server = http.createServer((req, res) => {
    res.writeHead(200, {
        'Content-Type': 'text/plain'
    });
    if (req.url === '/') {
        res.end('Welcome to Node.js');
    } else {
        res.writeHead(404);
        res.end('Page Not Found');
    }
});
server.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Output:

When the browser opens http://localhost:3000/:
Welcome to Node.js
When the browser opens http://localhost:3000/about:
Page Not Found

5. Common HTTP Methods

- **GET:** Retrieves data from the server.
- **POST:** Sends data to the server.
- **PUT:** Updates or replaces a resource.
- **DELETE:** Deletes a resource.

6. Real-Time Uses

HTTP request handling is used in login systems, web applications, REST APIs, online shopping websites, and form submissions.

Conclusion: Node.js handles HTTP requests through the built-in http module. The request object provides client request information, while the response object sends data back to the client.

Sending HTTP Requests in Node.js

Definition: Sending HTTP requests means sending a request from a Node.js application to another server to get or exchange data. Node.js uses the built-in http module for HTTP requests and the https module for secure HTTPS requests.

1. Methods of Sending Requests

1. http.get(): Sends a GET request to retrieve data from a server.
2. http.request(): Sends HTTP requests using different methods, such as GET, POST, PUT and DELETE.

2. Steps for Sending an HTTP Request

1. Import the built-in http module.
2. Specify the server URL and request options.
3. Send the request using http.get() or http.request().
4. Receive the response from the server.
5. Read the response data using the data event.
6. Process the complete response using the end event.
7. Handle errors using the error event.

3. Example of Sending an HTTP GET Request

```
const http = require('http');
http.get('http://localhost:3000/', (res)
    let data = '';
    res.on('data', (chunk) => {
        data += chunk;
    });
    res.on('end', () => {
        console.log('Response:', data);
    });
}).on('error', (err) => {
    console.log('Error:', err.message);
});
```

Output:
If a server is running on port 3000 and sends the response Welcome to Node.js, the output will be:

Response: Welcome to Node.js

4. Explanation of the Example

1. require('http') imports the HTTP module.
2. http.get() sends a GET request to the server.
3. The data event receives response data in chunks.
4. The end event runs when the complete response has been received.
5. The error event handles errors that occur during the request.

5. Advantages of Sending HTTP Requests

1. It allows communication between different servers.
2. It helps retrieve data from web services and APIs.
3. It supports data exchange between client and server applications.
4. It can be used to develop web applications and REST APIs.

Conclusion: Node.js provides the http.get() and http.request() methods to send HTTP requests. These methods allow applications to communicate with servers and receive data asynchronously.

Reading and Writing a File in Node.js

Definition: Node.js provides the built-in fs (File System) module to perform file operations, such as reading data from a file and writing data into a file. It supports both synchronous and asynchronous file operations.

1. Reading a File: The `fs.readFile()` method is used to read the contents of a file asynchronously.

Syntax: `fs.readFile(path, encoding, callback);`

Example:

```
const fs = require('fs');
fs.readFile('demo.txt', 'utf8', (err, data) => {
    if (err) {
        console.log(err);
        return;
    }
    console.log(data);
});
```

Output:
If demo.txt contains Hello Node.js, the output will be:
Hello Node.js

2. Writing a File: The `fs.writeFile()` method is used to write data into a file. If the file does not exist, Node.js creates it. If the file already exists, its previous contents are replaced.

Syntax: `fs.writeFile(path, data, callback);`

Example:

```
const fs = require('fs');
fs.writeFile('demo.txt', 'Welcome to Node.js', (err) => {
    if (err) {
        console.log(err);
        return;
    }
    console.log('File written successfully');
});
```

Output:
File written successfully
The file demo.txt will contain:
Welcome to Node.js

3. Common Methods of the File System Module

1. `fs.readFile()` : Reads data from a file asynchronously.
2. `fs.readFileSync()` : Reads data from a file synchronously.
3. `fs.writeFile()` : Writes data to a file asynchronously.
4. `fs.writeFileSync()` : Writes data to a file synchronously.
5. `fs.appendFile()` : Adds data to the end of a file.
6. `fs.unlink()` : Deletes a file.

4. Advantages

1. It allows applications to store and retrieve data.
2. It supports creating and modifying files.
3. It provides synchronous and asynchronous methods.
4. It is useful for logs, configuration files and data storage.
5. It is included in Node.js and does not require an external package.

Conclusion: The fs module is used to read and write files in Node.js. The readFile() method reads file contents, while writeFile() creates a file or replaces its existing contents with new data.

Writing a File Asynchronously in Node.js

Definition: Writing a file asynchronously means writing data into a file without blocking the execution of other operations. Node.js provides the fs.writeFile() method in the built-in File System (fs) module to write files asynchronously.

1. Syntax: `fs.writeFile(path, data, callback);`

- **path:** Name or path of the file.
- **data:** Content to be written into the file.
- **callback:** Function executed after the write operation completes. It receives an error if the operation fails.

2. Example

```
const fs = require('fs');
fs.writeFile('demo.txt', 'Hello Node.js', (err) => {
   if (err) {
      console.log(err);
      return;
   }
   console.log('File written successfully');
});
console.log('Program continues executing');
```

3. Output

```
Program continues executing
File written successfully
```

The callback output appears after the write operation completes. The exact timing of asynchronous operations depends on execution conditions.

The file demo.txt will contain:

```
Hello Node.js
```

4. Explanation

1. The fs module is imported using require('fs').
2. The fs.writeFile() method starts writing data into demo.txt.
3. If the file does not exist, Node.js creates it.
4. If the file already exists, its previous contents are replaced.
5. The callback function executes when the operation completes.
6. If an error occurs, the error is handled using err.
7. Other operations can continue while the file-writing operation is in progress.

5. Advantages

1. It does not block the main JavaScript execution while waiting for the file operation.
2. It improves application responsiveness.
3. It supports error handling through a callback.
4. It is useful for writing logs, configuration data and application files.

Conclusion: The `fs.writeFile()` method is used to write files asynchronously in Node.js. It allows the application to continue executing other operations while the file-writing task is handled in the background.

Opening a File in Node.js

Definition: Node.js provides the built-in File System (fs) module to open files. The fs.open() method is used to open a file and obtain a file descriptor, which can be used for further file operations.

1. Syntax: `fs.open(path, flags, callback);`

- path: Name or path of the file.
- flags: Specifies how the file should be opened.
- callback: Function that receives an error and the file descriptor.

2. Common File Opening Flags

1. `r`: Opens an existing file for reading.
2. `r+`: Opens an existing file for reading and writing.
3. `w`: Opens a file for writing and creates it if it does not exist. Existing contents are truncated.
4. `w+`: Opens a file for reading and writing, creating it if necessary and truncating existing contents.
5. `a`: Opens a file for appending and creates it if it does not exist.

3. Example

```
const fs = require('fs');
fs.open('demo.txt', 'r', (err, fd) => {
    if (err) {
        console.log('Error opening file:', err.message);
        return;
    }
    console.log('File opened successfully');
    console.log('File descriptor:', fd);
    fs.close(fd, (err) => {
        if (err) {
            console.log(err.message);
            return;
        }
        console.log('File closed successfully');
    });
});
```

4. Output:

If demo.txt exists, the output will be similar to:

```
File opened successfully
        File descriptor: 18
File closed successfully
```

Note: The file descriptor number may differ. If the file does not exist when using the r flag, an error occurs.

5. Explanation

1. The fs module is imported into the program.
2. The `fs.open()` method opens demo.txt in read mode.
3. If an error occurs, the callback handles it.
4. If successful, Node.js returns a file descriptor (fd).
5. The file descriptor can be used for further file operations.
6. The `fs.close()` method closes the file and releases the associated resource.

6. Advantages

1. It allows files to be opened with different access modes.
2. It provides a file descriptor for further operations.
3. It supports asynchronous file handling.
4. It helps manage file resources efficiently.

Conclusion: The `fs.open()` method opens a file using a specified flag and returns a file descriptor. The fs.close() method should be used when the file is no longer needed.

Deleting a File in Node.js

Definition: Node.js provides the built-in File System (fs) module to perform file operations. The fs.unlink() method is used to delete a file from the file system.

1. Syntax: fs.unlink(path, callback);

- path: Name or path of the file to be deleted.
- callback: Function executed after the deletion operation completes. It receives an error if the operation fails.

2. Example

```
const fs = require('fs');
fs.unlink('demo.txt', (err) => {
    if (err) {
        console.log('Error deleting file:', err.message);
        return;
    }
    console.log('File deleted successfully');
});
```

3. Output

If demo.txt exists and is successfully deleted:
File deleted successfully
If the file does not exist, an error message will be displayed.

4. Explanation

1. The fs module is imported using require('fs').
2. The fs.unlink() method specifies the file to be deleted.
3. Node.js performs the deletion asynchronously.
4. If an error occurs, the callback receives the error object.
5. If the operation succeeds, a success message is displayed.
6. After successful deletion, the file is no longer available at that path.

5. Advantages

1. It allows files to be deleted programmatically.
2. It supports asynchronous file operations.
3. It provides error handling through a callback.
4. It is useful for removing temporary files and unnecessary application data.

Conclusion: The fs.unlink() method is used to delete files in Node.js. It performs the operation asynchronously and provides error information through its callback function.

Other I/O Operations in Node.js: Append, Rename and Truncate

Definition: Node.js provides the built-in File System (fs) module to perform various input/output (I/O) operations on files. Three important operations are appending data, renaming files and truncating file contents.

1. Append a File: The fs.appendFile() method is used to add new data at the end of an existing file. If the file does not exist, Node.js creates it.

Syntax:
```
fs.appendFile(path, data, callback);
```

Example:
```
const fs = require('fs');
fs.appendFile('demo.txt', ' Welcome to Node.js', (err) => {
    if (err) {
        console.log(err);
        return;
    }
    console.log('Data appended successfully');
});
```

Output: Data appended successfully
If the file originally contains Hello, it will contain Hello Welcome to Node.js after the operation.

2. Rename a File: The fs.rename() method is used to change the name or move a file from one location to another.

Syntax:
```
fs.rename(oldPath, newPath, callback);
```

Example:
```
const fs = require('fs');
fs.rename('demo.txt', 'newdemo.txt', (err) => {
    if (err) {
        console.log(err);
        return;
    }
    console.log('File renamed successfully');
});
```

Output:
File renamed successfully
The file demo.txt is renamed to newdemo.txt.

3. Truncate a File: The fs.truncate() method is used to reduce the size of a file to a specified length. If the length is set to zero, the file becomes empty but is not deleted.

Syntax:
```
fs.truncate(path, len, callback);
```

- **path:** Name or path of the file.
- **len:** Desired file length in bytes.
- **callback:** Function executed after the operation completes.

Example:
```
const fs = require('fs');
fs.truncate('demo.txt', 0, (err) => {
    if (err) {
        console.log(err);
        return;
    }
    console.log('File truncated successfully');
});
```

Output:
File truncated successfully
After execution, demo.txt remains available but contains no data.

Conclusion: The Node.js fs module provides fs.appendFile() to add data, fs.rename() to rename or move a file, and fs.truncate() to reduce its size. These operations help manage files efficiently in Node.js applications.

1. File System Module with URL Module: Node.js provides the built-in fs (File System) module to perform file and directory operations. The built-in url module is used to parse and handle URLs. These modules can be used together to work with files whose paths are obtained from URLs.

Important Methods:

1. fs.readFile() : Reads the contents of a file.
2. fs.writeFile() : Writes data into a file.
3. fs.mkdir() : Creates a directory.
4. fs.readdir() : Reads the contents of a directory.
5. fs.rm() : Removes a file or directory.
6. url.parse() : Parses a URL into its components. This is a legacy API; modern Node.js applications generally use the URL class.

Example:

```
const fs = require('fs');
const { URL } = require('url');
const myURL = new URL(
    'http://localhost:3000/files/demo.txt'
);
console.log('Hostname:', myURL.hostname);
console.log('Pathname:', myURL.pathname);
fs.readFile('demo.txt', 'utf8', (err, data) => {
    if (err) {
        console.log(err.message);
        return;
    }
    console.log('File content:', data);
});
```

Conclusion: The fs and url modules can be used together to handle URLs and perform file operations in Node.js.

File System Module with URL Module and Directory Operations in Node.js-1

Explanation:

1. The fs module is imported to access file operations.
2. The URL class is imported from the url module.
3. The URL object provides the hostname and pathname.
4. The fs.readFile() method reads the contents of demo.txt.
5. Errors are handled through the callback function.

2. Creating a Directory: The fs.mkdir() method is used to create a new directory in the file system.

Syntax: `fs.mkdir(path, callback);`

Example:

```
const fs = require('fs');
fs.mkdir('myfolder', (err) => {
    if (err) {
        console.log(err.message);
        return;
    }
    console.log('Directory created successfully');
});
```

Output:
Directory created successfully

Explanation:

1. The fs module is imported.
2. The fs.mkdir() method creates the directory named myfolder.
3. If an error occurs, the callback handles it.
4. If successful, a confirmation message is displayed.

3. Reading a Directory: The fs.readdir() method is used to read the names of files and subdirectories contained in a directory.

Syntax: `fs.readdir(path, callback);`

Example:

```
const fs = require('fs');
fs.readdir('myfolder', (err, files) => {
    if (err) {
        console.log(err.message);
        return;
    }
    console.log('Directory contents:', files);
});
```

Output:
If the directory contains a.txt and b.txt, the output will be:
Directory contents: ['a.txt', 'b.txt']

Explanation:

1. The fs.readdir() method specifies the directory to read.
2. The callback receives an error or an array of entry names.
3. The array is displayed using console.log().
4. The method does not read the contents of the individual files.

4. Removing a Directory

Definition:

The fs.rmdir() method can remove an empty directory. For general directory removal in modern Node.js, fs.rm() can also be used.

Syntax: `fs.rmdir(path, callback);`

Example:

```
const fs = require('fs');
fs.rmdir('myfolder', (err) => {
    if (err) {
        console.log(err.message);
        return;
    }
    console.log('Directory removed successfully');
});
```

Output: Directory removed successfully

File System Module with URL Module and Directory Operations in Node.js-2

Explanation:

1. The fs.rmdir() method specifies the directory to remove.
2. The directory must be empty for this example to succeed.
3. If an error occurs, the callback handles it.
4. If successful, the directory is removed.

Note: In modern Node.js, fs.rm('myfolder', { recursive: true }, callback) can remove a directory and its contents. Use recursive removal carefully because it can permanently delete files and subdirectories.

Conclusion: The fs module manages files and directories, while the url module parses and handles URLs. Node.js provides fs.mkdir() to create directories, fs.readdir() to read directory contents, and fs.rmdir() to remove empty directories.